SE LAB 4

Name: Shreya Shenoy
SRN: PES2UG24CS487
Class: 5H
Date: 30-9-2026

Game assigned: Whack-a-Mole

Used tool: Github Copilot
Number of commits made: 5
Github link:
Prompt 1:
I am a gamer and a Python developer working on an existing Whack-a-Mole game that I want to improve and make more complete and polished.

The project is a simple desktop Whack-a-Mole game developed using Python and the Pygame library.
The existing project contains main.py, README.md, and requirements.txt.
Please inspect the existing code and README before making any changes.
The current implementation should be treated as the base version, so preserve its existing gameplay, visual style, and overall structure wherever possible.
My objective is to improve the existing game by fixing the identified gameplay issue and adding the features specified in the README.
I do not want the project unnecessarily rewritten or redesigned.
Make focused modifications to the existing implementation rather than replacing it with a completely different game.
The first required improvement is to fix the mole collision/click detection.
Currently, a single mouse click can potentially register multiple hits when moles are positioned close to each other.
A single click should result in at most one successful hit and one score increment.
The collision logic should therefore correctly determine which mole was actually clicked and prevent duplicate scoring from the same click.

Prompt 2:

The second requirement is to implement a proper Game Over state.
When the game timer reaches zero, gameplay should stop and the player should clearly see a Game Over screen. The Game Over . the word must be large and visible and have a white border. The font must look

Sci-fi vintage game like
The Game Over screen should display the player's final score and provide an appropriate way to continue or exit with the replay symbol button saying retry? or end game.

Prompt 3:
The third requirement is to add a replay which i already added in the previous requirement, but if betterment is possible please do to make it render better and also add difficulty-selection system.
After the game ends, the player should be able to choose between Easy, Medium, and Hard difficulty modes or exit the game.
The selected difficulty should meaningfully affect gameplay, such as mole appearance/spawn timing or another suitable gameplay parameter.
The implementation should remain simple, understandable, and consistent with the existing game.
The difficulty depends on the number of moles appearing at a given time in the game and also the speed of the moles.
say easy level has 2 moles and speed is normal , then difficulty is hard would be say 7 moles and extreme speed of 3x

also what i forgot to mention was , the technology stack must remain Python and Pygame.
Do not introduce unnecessary external frameworks or libraries.
Keep the code readable, modular, and beginner-friendly.
Use the existing requirements.txt where appropriate and add dependencies only if genuinely necessary.
While implementing the changes, inspect the existing code carefully and reuse existing functions, variables, and game logic wherever practical.
Avoid unnecessary changes to the UI, graphics, controls, or gameplay mechanics that are not required by the assignment.

Prompt 4:
The fourth requirement is to add sound effects.
The game should provide separate audio feedback for a successful mole i want a twinkle, a missed click must have a boink sound , and the Game Over event default sound if available or the sound effects closest to this

Use Pygame-compatible audio and handle the sounds cleanly without breaking the game if an audio asset needs to be loaded or configured.

Final prompt:
the final output is the sanity check , I want is a fully working modified version of the existing Whack-a-Mole game with all my four requested improvements that should work together without interfering with one another.
The game should launch normally using the existing project setup and should not produce runtime errors and must run smoothly.

After making the changes, explain what files were modified, what was changed in each file, how each requirement was implemented, and how I can run and test the completed game.
Also identify any new assets or files that were added and explain where they should be placed.

Before finalizing, review the implementation for obvious bugs, especially duplicate scoring, timer/game-state transitions, difficulty selection, replay behavior, mouse-click handling, and sound playback.
Do not modify README.md unless it is necessary to document the implemented changes.

Videos of before and after added to the repo